

# How Much Does a Heuristic Buy?

## A\* versus Branch and Bound for Least-Effort Walking Routes on Terrain Maps

Abdulla Alfalasi

COM1005 coursework, University of Sheffield (May 2021)

<https://github.com/aalbudoor/astar-vs-branch-and-bound>

### Abstract

A Rambler crosses a height map one cell at a time. Level or downhill steps cost 1, and climbing costs 1 plus the height gained. We compare branch and bound (uniform-cost search) with A\* under five heuristics on the 16×16 course map and three seeded 64×64 synthetic terrains, using 500 random start/goal pairs in total. Four heuristics never overestimate the remaining cost: straight-line distance, Manhattan distance, remaining ascent, and Manhattan distance plus remaining ascent. The fifth, absolute height difference, can overestimate it. All four admissible heuristics returned an optimal route in every run. The combined heuristic did best everywhere. It expanded 25% of the nodes branch and bound expanded on smooth terrain, but 72% on rugged terrain, and a simple informedness measure,  $h(\text{start})/C^*$ , tracks this gap. The absolute-height heuristic returned a suboptimal route in 22–56% of pairs. On the two noisier 64×64 maps it also expanded more nodes than branch and bound, because it repeatedly reopened closed nodes.

## 1 Introduction

Path planning over terrain is a standard testbed for informed search. The textbook claim is that A\* with an admissible heuristic finds the same optimal path as an uninformed method such as branch and bound or Dijkstra’s algorithm [2, 4], while expanding fewer nodes [1, 3]. How many fewer depends on how much of the true cost the heuristic captures, and that in turn depends on the terrain.

This report asks three questions:

1. How many node expansions does each heuristic save relative to branch and bound, and how does this change with the terrain?
2. Does combining distance and ascent information help, and can the gain be predicted from the heuristic alone?
3. What does an inadmissible heuristic cost in route quality and in search effort?

## 2 Problem

A terrain map is a grid of heights  $H(r, c) \in \{0, \dots, 255\}$ . A state is a cell  $(r, c)$ , and the Rambler can move to any of its four neighbours. The cost of stepping from cell  $u$  to a neighbouring cell  $v$  is

$$c(u, v) = \begin{cases} 1 & \text{if } H(v) \leq H(u), \\ 1 + H(v) - H(u) & \text{if } H(v) > H(u), \end{cases} \quad (1)$$

Table 1: Heuristics compared. “Admissible” means the heuristic never exceeds the true remaining cost.

| Name                           | Definition                                         | Admissible |
|--------------------------------|----------------------------------------------------|------------|
| Euclidean $h_E$                | $\lfloor \sqrt{(r - r_g)^2 + (c - c_g)^2} \rfloor$ | yes        |
| Manhattan $h_M$                | $ r - r_g  +  c - c_g $                            | yes        |
| Ascent $h_A$                   | $\max(0, H(r_g, c_g) - H(r, c))$                   | yes        |
| Manhattan+Ascent $h_{M+A}$     | $h_M + h_A$                                        | yes        |
| Absolute height $h_{ \Delta }$ | $ H(r_g, c_g) - H(r, c) $                          | no         |

so descending is as cheap as walking on the flat and every metre climbed is paid for. The task is to find a path of least total cost  $C^*$  from a start cell to a goal cell.

## 3 Methods

### 3.1 Search algorithms

Both algorithms keep an *open* list of generated nodes and a *closed* list of expanded nodes. A node records its state, its cost so far  $g$ , and its parent. When a successor reaches a state already on open or closed by a cheaper route, that node is updated and, if it was closed, moved back to open. Search stops when a goal node is *selected* for expansion, not when it is generated, which is what makes the returned path optimal.

**Branch and bound** always selects the open node with the lowest  $g$ . With the duplicate check above it is uniform-cost search, the node-selection form of Dijkstra’s algorithm [2, 6].

**A\*** selects the open node with the lowest  $f = g + h$ , where  $h$  estimates the remaining cost to the goal [3]. With  $h \equiv 0$  it reduces to branch and bound. We checked this in the code: on every instance tested, A\* with  $h \equiv 0$  reproduced branch and bound’s path cost and node count exactly.

### 3.2 Heuristics

Let  $(r, c)$  be the current cell and  $(r_g, c_g)$  the goal. Table 1 lists the five heuristics compared.

**Proposition 1.**  $h_{M+A}$  is admissible and consistent, and it dominates  $h_M$ ,  $h_E$  and  $h_A$ .

*Proof.* Write each step cost in (1) as  $1 + \max(0, \Delta)$ , where  $\Delta$  is the height change of the step. Any path from  $u$  to the goal has at least  $h_M(u)$  steps. Its positive height changes sum to at least the net rise  $H(\text{goal}) - H(u)$ , and to at least 0. The path therefore costs at least  $h_M(u) + h_A(u) = h_{M+A}(u)$ , which proves admissibility. For consistency, take one step from  $u$  to  $v$ . Then  $h_M(u) \leq 1 + h_M(v)$ . The inequality  $\max(0, a + b) \leq \max(0, a) + \max(0, b)$  gives  $h_A(u) \leq \max(0, H(v) - H(u)) + h_A(v)$ . Adding the two gives  $h_{M+A}(u) \leq c(u, v) + h_{M+A}(v)$ . Dominance holds because  $h_A \geq 0$ ,  $h_M \geq 0$  and  $h_E \leq h_M$ .  $\square$

$h_{|\Delta|}$  looks like a natural “height difference” heuristic, but it also charges for descents, which are cheap: a single downhill step of 100 costs 1, yet  $h_{|\Delta|}$  estimates it at 100. By standard results, a consistent heuristic that dominates another expands no more nodes, ignoring ties [1, 5]. We

Table 2: Terrain maps. Synthetic maps are regenerated from the listed seed by `TerrainGenerator`.

| Map                             | Size  | Hills | Hill width $\sigma$ | Noise s.d. | Pairs |
|---------------------------------|-------|-------|---------------------|------------|-------|
| <code>tmc16</code> (course map) | 16×16 | —     | —                   | —          | 200   |
| <code>smooth64</code>           | 64×64 | 8     | 8–16                | 0          | 100   |
| <code>rolling64</code>          | 64×64 | 30    | 3–8                 | 4          | 100   |
| <code>rugged64</code>           | 64×64 | 120   | 1.5–4               | 12         | 100   |

therefore expect  $h_{M+A}$  to expand the fewest nodes of the admissible heuristics. No such guarantee applies to  $h_{|\Delta|}$ .

## 4 Experimental Setup

**Maps.** We use the 16×16 map `tmc.pgm` supplied with the coursework, and three 64×64 maps generated as sums of Gaussian hills with per-cell Gaussian noise, rescaled to 0–255 (Table 2). The three maps range from a few broad hills to many narrow, noisy ones.

**Instances.** For each map we sampled start/goal pairs uniformly at random with a fixed seed. We kept only pairs at least half the map width apart in Manhattan distance, which excludes trivial instances. Every method ran on the same pairs.

**Metrics.** *Nodes expanded* counts every selection from open, including re-expansions of reopened nodes. We report it relative to branch and bound on the same pair and average the ratio over pairs. *Optimality* compares each path’s cost with branch and bound’s, which is optimal. *Informedness* is  $h(\text{start})/C^*$ , the fraction of the true cost the heuristic sees before the search begins. We also give the coursework’s *efficiency* measure, path length divided by nodes expanded. Wall-clock time is recorded, but open and closed are linear-scan lists, so time mostly tracks node count.

**Implementation.** The search engine (`Search`, `SearchNode`, `SearchState`, `TerrainMap`) is the COM1005 base code by Phil Green and Heidi Christensen. The Ramblers state, the heuristics, the terrain generator and the experiment harness are ours. The code is Java 8 compatible, and `run_experiments.sh` regenerates every number and figure in this report. Apart from timing, the results are deterministic.

## 5 Results

### 5.1 Search effort and optimality

Table 3 and Figure 1 summarize the 500 pairs. All four admissible heuristics returned an optimal route on every pair of every map, as the theory predicts. Their savings differ widely.

Manhattan+Ascent was the best heuristic on every map. On `smooth64` it expanded a quarter of the nodes branch and bound expanded, and its mean search time fell from 22.4 ms to 2.0 ms in our run (timings vary from run to run; node counts do not). Distance alone mattered more than ascent alone on the synthetic maps. On the small course map the two did equally well (0.79), and adding them together gave far more than either alone. The ordering  $h_{M+A} < h_M < h_E$  in nodes expanded matches the dominance ordering of the three heuristics.

Table 3: Mean nodes expanded relative to branch and bound (lower is better), percentage of pairs solved optimally, and mean and maximum cost excess over the optimum. Branch and bound’s own mean expansions are 148, 2360, 2622 and 2797 for the four maps.

| Method                                                     | tmc16       |        | smooth64    |        | rolling64   |        | rugged64    |        |
|------------------------------------------------------------|-------------|--------|-------------|--------|-------------|--------|-------------|--------|
|                                                            | ratio       | opt. % | ratio       | opt. % | ratio       | opt. % | ratio       | opt. % |
| Branch and bound                                           | 1.00        | 100    | 1.00        | 100    | 1.00        | 100    | 1.00        | 100    |
| A* Euclidean                                               | 0.83        | 100    | 0.57        | 100    | 0.74        | 100    | 0.85        | 100    |
| A* Manhattan                                               | 0.79        | 100    | 0.45        | 100    | 0.66        | 100    | 0.81        | 100    |
| A* Ascent                                                  | 0.79        | 100    | 0.82        | 100    | 0.81        | 100    | 0.91        | 100    |
| A* Manhattan+Ascent                                        | <b>0.56</b> | 100    | <b>0.25</b> | 100    | <b>0.48</b> | 100    | <b>0.72</b> | 100    |
| A* Abs. height                                             | 0.76        | 71     | 0.71        | 78     | 1.11        | 44     | 1.10        | 63     |
| Cost excess of A* Abs. height over optimum: mean / max (%) |             |        |             |        |             |        |             |        |
|                                                            | 16.9 / 455  |        | 2.7 / 51    |        | 7.5 / 126   |        | 1.9 / 22    |        |

Table 4: Mean informedness  $h(\text{start})/C^*$  of each heuristic. Values above 1 mean the heuristic overestimates.

| Heuristic        | tmc16 | smooth64 | rolling64 | rugged64 |
|------------------|-------|----------|-----------|----------|
| Euclidean        | 0.20  | 0.49     | 0.27      | 0.15     |
| Manhattan        | 0.27  | 0.64     | 0.35      | 0.20     |
| Ascent           | 0.24  | 0.15     | 0.13      | 0.07     |
| Manhattan+Ascent | 0.51  | 0.79     | 0.48      | 0.27     |
| Abs. height      | 1.72  | 0.63     | 0.44      | 0.16     |

## 5.2 Why the savings shrink on rugged terrain

The gain from every heuristic fell as the terrain became rougher: Manhattan+Ascent went from 0.25 on **smooth64** to 0.72 on **rugged64**. Table 4 shows why. On smooth terrain the optimal route is close to a straight walk plus the unavoidable climb, so  $h_{M+A}$  sees 79% of the true cost from the start. On rugged terrain the optimal route pays for many small climbs and detours that no local estimate can see, and  $h_{M+A}$  sees only 27%. Across the three synthetic maps, which share a size, lower informedness goes with a higher expansion ratio for every admissible heuristic except Ascent, whose informedness is low everywhere. Unlike the expansion ratio, informedness can be measured from a sample of solved instances without comparing search runs.

## 5.3 The cost of an inadmissible heuristic

The absolute-height heuristic returned a suboptimal route on 22–56% of pairs. The worst route on the course map cost 61 against an optimum of 11, 5.5 times as much. It was also not reliably faster. On **rolling64** and **rugged64** it expanded about 10% *more* nodes than branch and bound, and on one **rolling64** pair it expanded 20,047 nodes on a 4,096-cell map. An inconsistent heuristic can close a node by an expensive route and then find a cheaper one. The node is then reopened and expanded again, and those re-expansions add up. On the course map it has a mean informedness of 1.72, so it usually overestimates, and its apparent savings there come largely from that overestimation. It prunes aggressively by ignoring routes that are actually cheap.

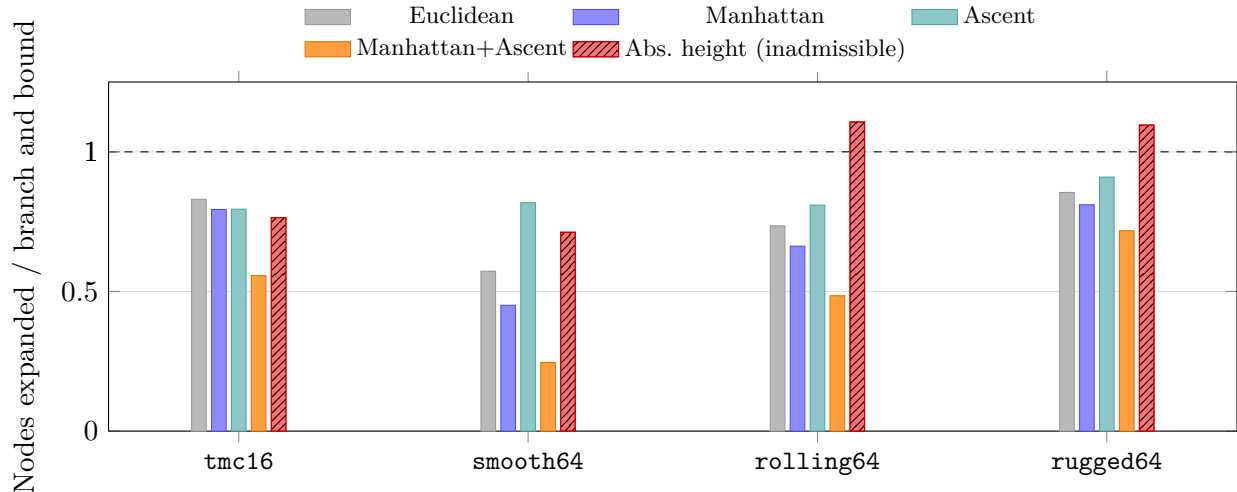


Figure 1: Mean nodes expanded by A\* relative to branch and bound (dashed line). Manhattan+Ascent expands the fewest nodes on every map. The inadmissible absolute-height heuristic does worse than branch and bound on the two noisier 64×64 maps.

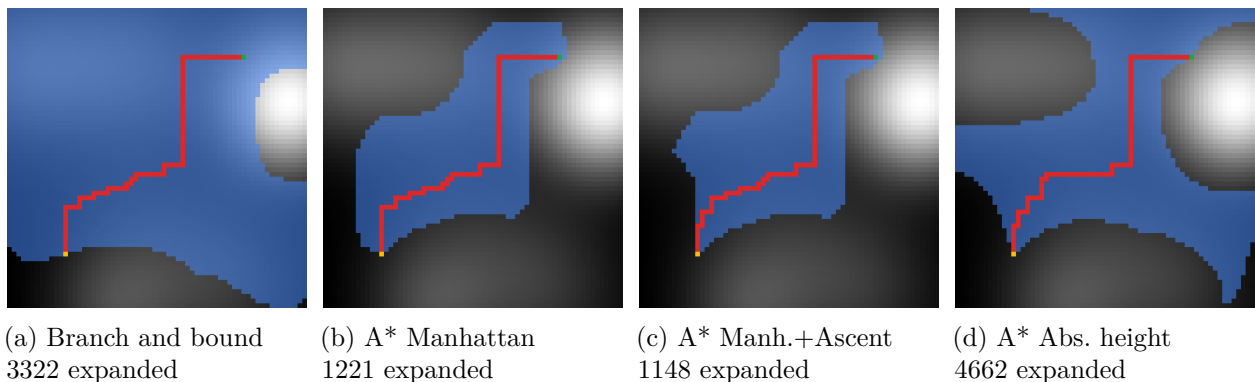


Figure 2: Start (green, top right) to goal (yellow, bottom left) on **smooth64**; lighter cells are higher. Blue cells were expanded and the red line is the route found. All four routes cost 95.

## 5.4 Example

Figure 2 shows one pair on **smooth64**. All four methods returned routes of cost 95. Branch and bound expanded 3322 nodes, most of the map. Manhattan expanded 1221 and Manhattan+Ascent 1148, both confined to a corridor around the route. The absolute-height heuristic still found the optimum here but expanded 4662 nodes, more than the map contains, because of re-expansions.

## 6 Discussion and Limitations

The practical lesson is to build the heuristic from the cost function. Each step in this problem pays for distance and for climbing, and the heuristic that bounds both terms beat the two that bound only one. Informedness is a cheap diagnostic: it can be computed from a sample of solved instances before choosing a heuristic, and on the synthetic maps it tracked the terrain effect.

The study has several limitations. The synthetic maps come from one generator with one

seed per roughness level, so the terrain effect is shown on three maps, not a distribution of them. Movement is 4-connected, and diagonal moves would change both the costs and which heuristics are admissible. Open and closed are linear lists, as in the base code, so timings would shrink with a priority queue and a hash set. The node-count comparisons would not change. Ties in  $f$  are broken by list order, which can shift node counts slightly between heuristics with equal  $f$  values.

## 7 Conclusion

On least-effort terrain routing, A\* with any admissible heuristic returned the same optimal routes as branch and bound, and it expanded between 25% and 91% as many nodes, depending on the heuristic and the terrain. A heuristic that combines Manhattan distance with remaining ascent is admissible and consistent, and it was the best choice on every map. Its advantage shrinks on rugged terrain, as its informedness predicts. The absolute-height heuristic is not admissible. It returned suboptimal routes on up to 56% of pairs, and on the two noisier 64×64 maps it searched more than branch and bound did.

## Acknowledgements

The search framework and terrain map reader are the COM1005 base code written by Phil Green (2013) and Heidi Christensen (2021) at the University of Sheffield.

## References

- [1] Rina Dechter and Judea Pearl. Generalized best-first search strategies and the optimality of A\*. *Journal of the ACM*, 32(3):505–536, 1985. doi: 10.1145/3828.3830.
- [2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- [4] Eugene L. Lawler and David E. Wood. Branch-and-bound methods: A survey. *Operations Research*, 14(4):699–719, 1966. doi: 10.1287/opre.14.4.699.
- [5] Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [6] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.